

Au-delà de la syntaxe

Architecture et méthodologies à l'ère du Vibe Coding

Raphaël Semeteys

28_05_2026 DEVCON#27

programmez!

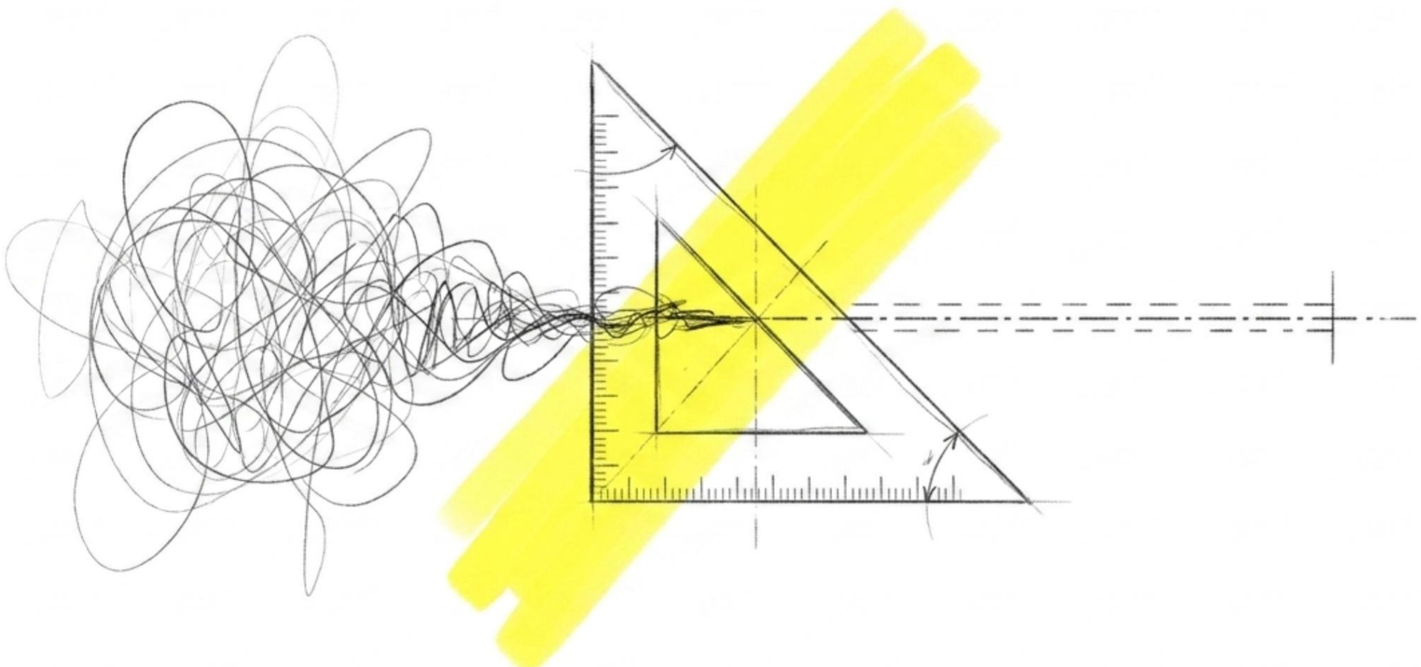
DELL Technologies

42

FIPARCO

DEVCON#27 : MERCI AUX PARTENAIRES DE LA DEVCON

Introduction



Raphaël Semeteys

Head of Tech Advocacy

Senior Architect

Raphiki Yogeek

Podcast Projets Libres

Prof de Yoga

« Ecrivain » à l’ère de l’IA



Vibe Coding ?



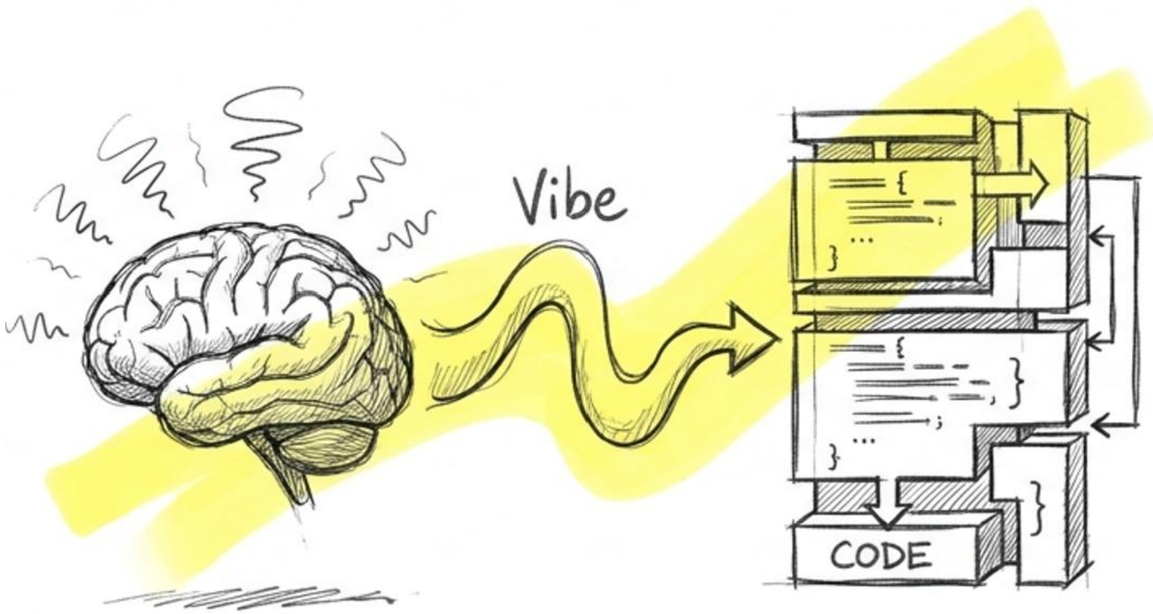
- Coder par l'**intuition** ou au ressenti



- Comment passer du projet jetable à la **production industrielle** ?



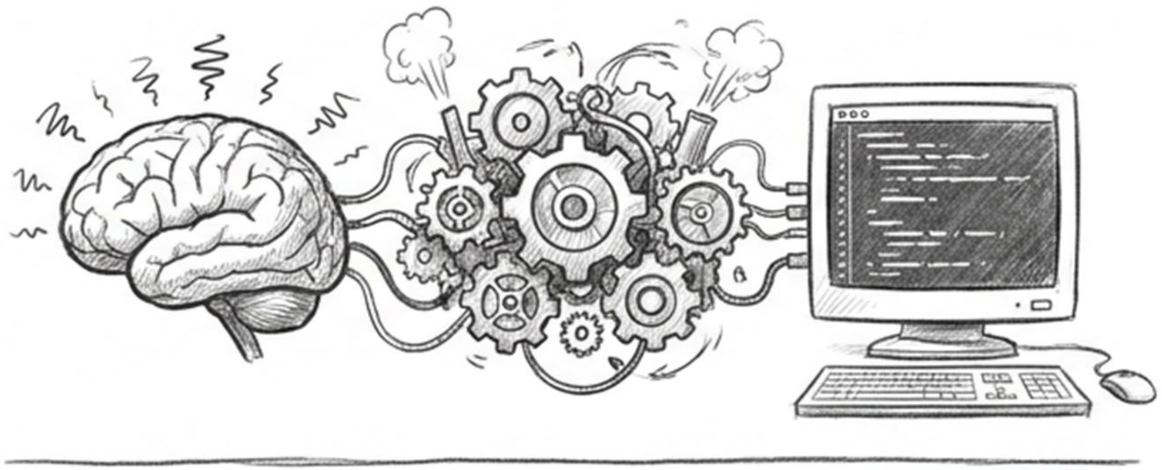
- L'ingéniérie des **spécifications** redevient une compétence clé



De la syntaxe à l’intention

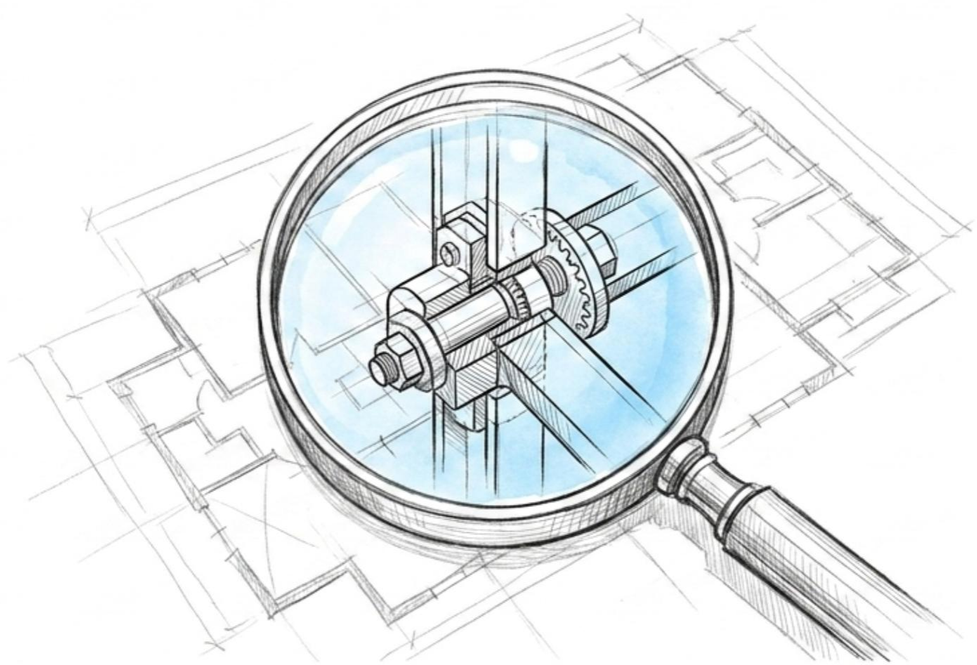
- **Avant** : traduction mentale en syntaxe
- **Aujourd’hui** : l’intention devient le langage
- **Nouveau rôle** : chef d’orchestre (Décrire → Observer → Ajuster)
- **Risques** : failles de sécurité et confiance paradoxale







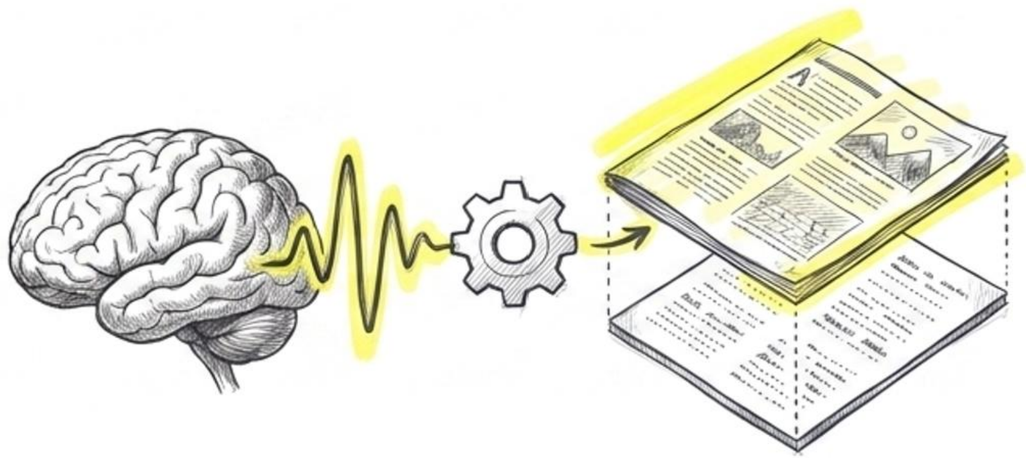
Exemples



Exemple 1 - Focus sur l’intention

Archiver un magazine Web

- **Défi** : automatiser une tâche linéaire et isolée
- **Méthode** : itérations rapides et conversationnelles ; focus sur l'intention plutôt que la syntaxe
- **Solution** : script généré pas à pas pour accomplir l'objectif (OCR, PDF Sandwich)
- **Bilan** : efficace pour des outils jetables, mais limité par l'absence de persistance

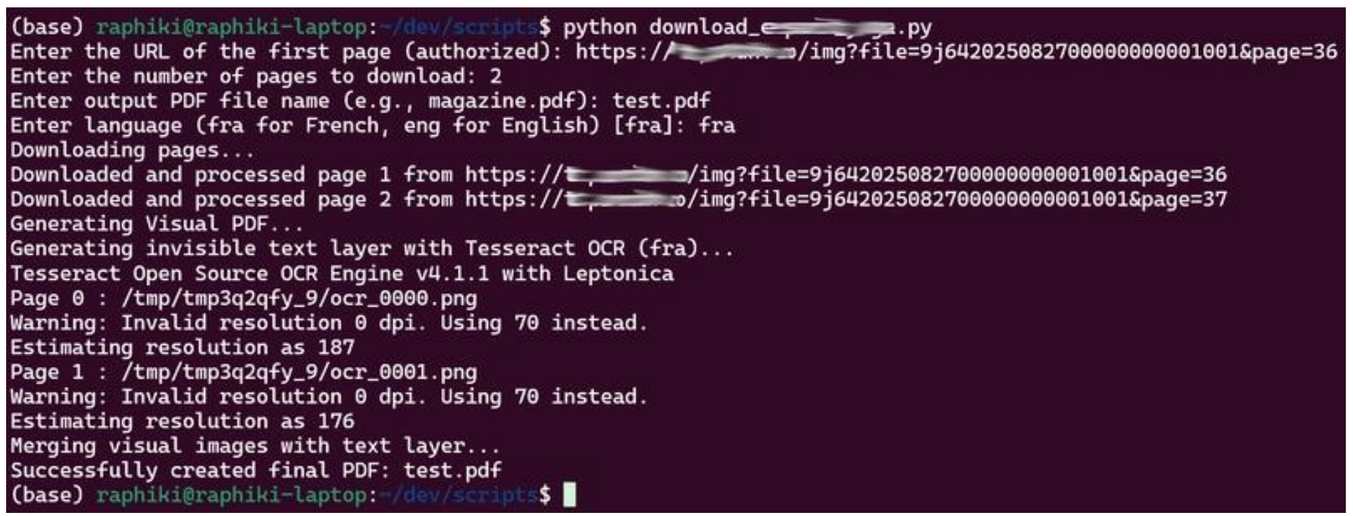


Exemple 1 - Focus sur l'intention

Archiver un magazine Web

Session de pur Vibe Coding





- **Script** Python ~100 lignes, `pillow` (pixels), `pypdf` (structure)
- **Traitement d'image** : `tesseract` (options avancées), resampling LANCZOS
- **Intention** au final : un PDF de qualité avec recherche

Exemple 2 - Spécifications explicites

Développer une application Web complète

- **Défis** : gestion UI avancée, amnésie sur des sessions longues
- **Méthode** : spec.md comme mémoire à long terme
- **Solution** : itérations visuelles guidées et résumés de session
- **Bilan** : l'IA gère la complexité si le contexte est solidement ancré



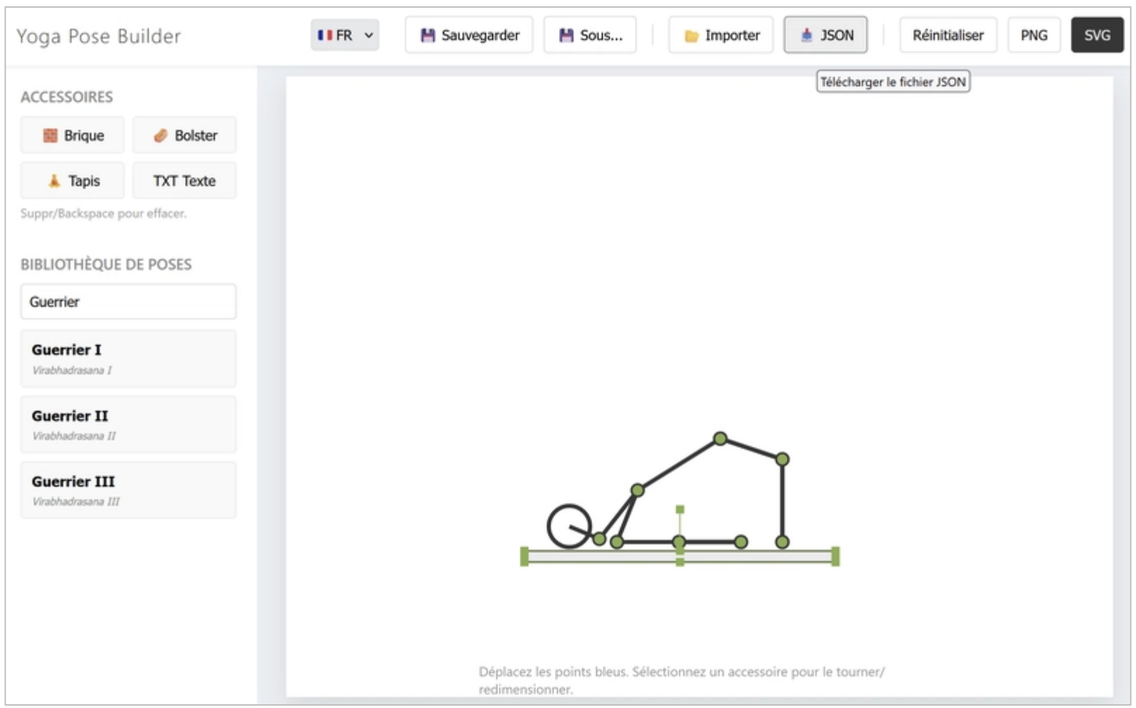
Exemple 2 - Spécifications explicites

Développer une application Web complète

Session hybride

Spec + Vibe Coding UI++

```
{
  "meta": { "nameFR": "Demi-Pont", "nameSK": "Setu Bandhasana" },
  "joints": {
    "head": { "x": -120, "y": 100 }, "neck": { "x": -100, "y": 100 }, "chest": {
    "lShoulder": { "x": -80, "y": 100 }, "lElbow": { "x": -20, "y": 100 }, "lHar
    "rShoulder": { "x": -80, "y": 100 }, "rElbow": { "x": -20, "y": 100 }, "rHar
    "lHip": { "x": 20, "y": 0 }, "lKnee": { "x": 80, "y": 20 }, "lFoot": { "x":
    "rHip": { "x": 20, "y": 0 }, "rKnee": { "x": 80, "y": 20 }, "rFoot": { "x":
  }
}
```

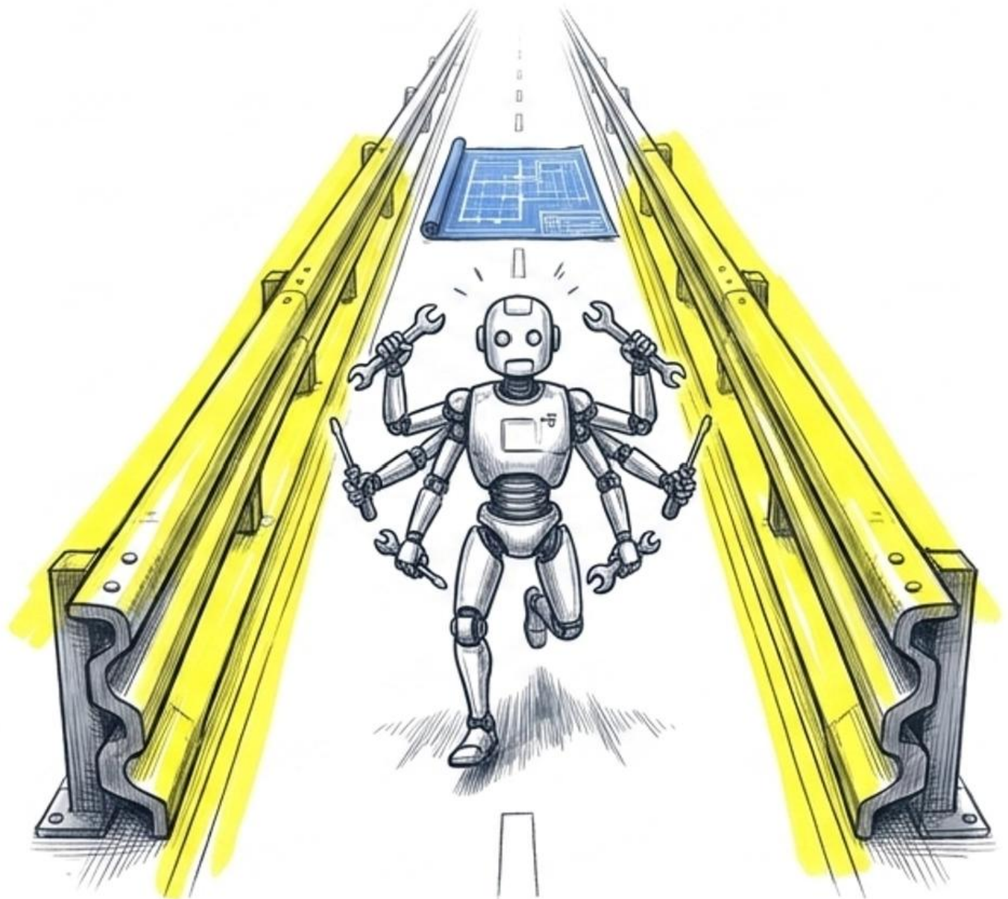


- **Doc** : `spec.md`, `features.md` et `session_summary.md`
- **Code** : Node/Express, Fabric.js proposé pour le rig du mannequin

Exemple 3 - Architecture explicite

Construire un système complexe

- **Défis** : éviter le code spaghetti, réduire les hallucinations
- **Méthode** : imposer un cadre d'architecture, cadrer un junior indiscipliné
- **Solution** : monolithe modulaire, ADRs, vérification par rétro-ingénierie
- **Bilan** : l'humain se concentre sur l'architecture ; contrôle via garde-fous stricts



Exemple 3 - Architecture explicite

Construire un système complexe

Session Spec-Driven Development

Focus important sur l'architecture

```
I am the Lead Architect. You are the Senior Developer.

Context Loading:
1. Read Docs/RULES.md (The Law).
2. Read Docs/TECH_CONTEXT.md (The Stack).
3. Read Docs/ADR/002-modular-monolith.md (The Blueprint).
4. Read Docs/Features/002-global-implementation-plan.md (The Plan).

Current State:
We are in Phase 4. Previous phases are frozen.

Task:
Implement the Logic Engine defined in Docs/Features/005-logical-engine-specifica
Constraint:
Do not touch /apps/web yet. Focus on /packages/shared.
```

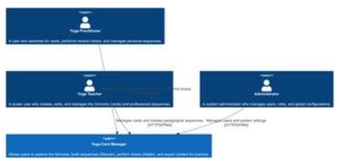
```
45   ## Project Status: ROADMAP COMPLETE
46   All five phases of the initial implementation plan are now verified and complete.

✦ I have updated the session summary to include the reference to the new PWA ADR.

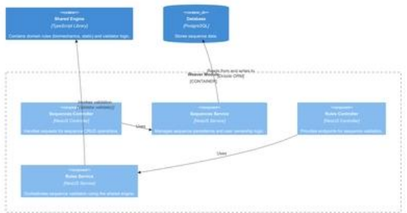
  The project is complete.

  To Restore Context for Next Session:
  Please read docs/TECH_STATE_SNAPSHOT.md and docs/archive/session-003-pdf-generation-summary.md.

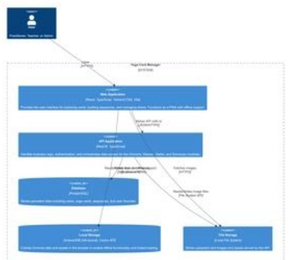
  Thank you for building the YogĀrkana Codex with me! Namaste. 🙏
```



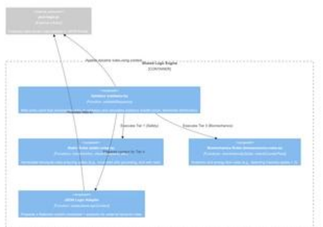
C4 Level 1 – System Context



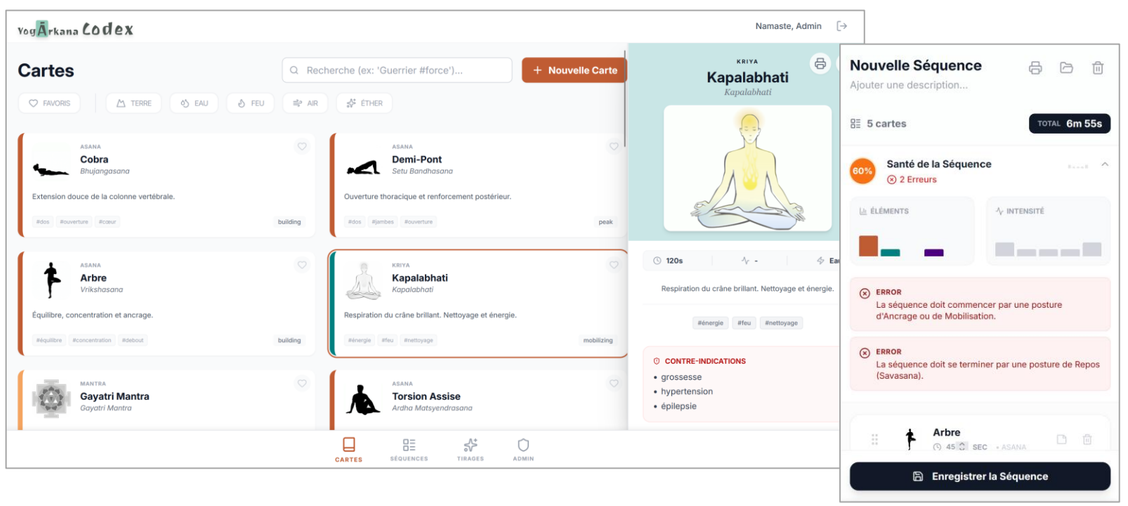
C4 Level 3 – Component (Weaver Module)



C4 Level 2 – Container



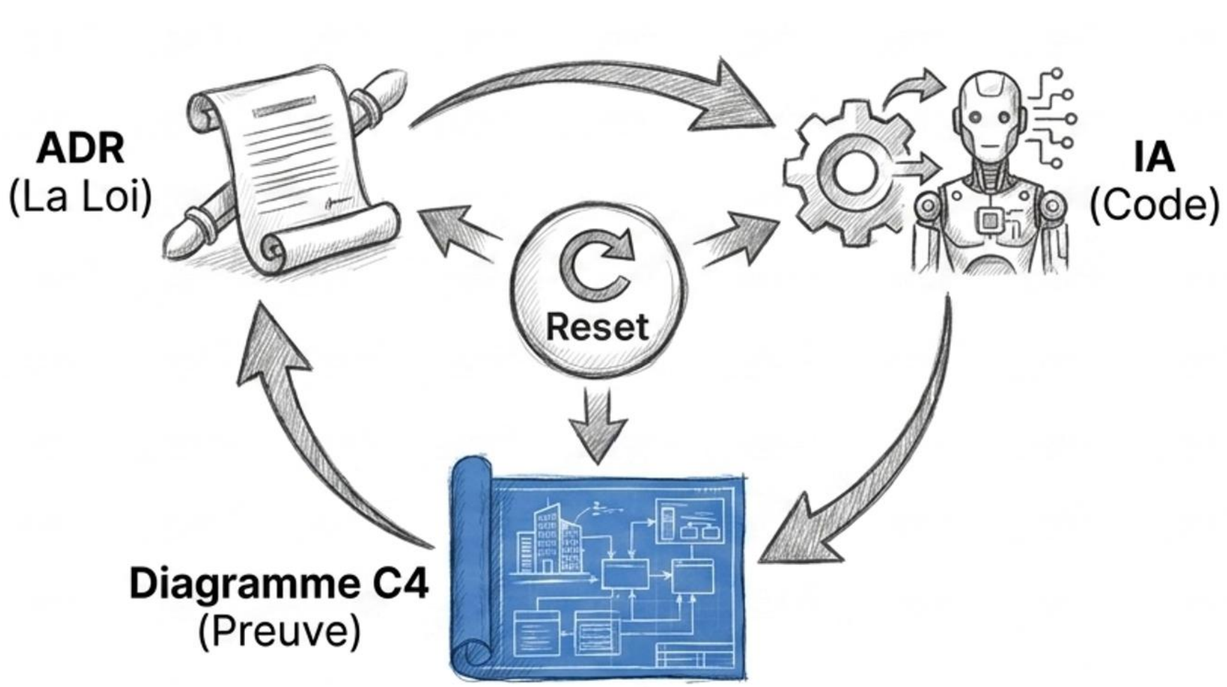
C4 Level 4 – Code (Logic Engine)



programmez!

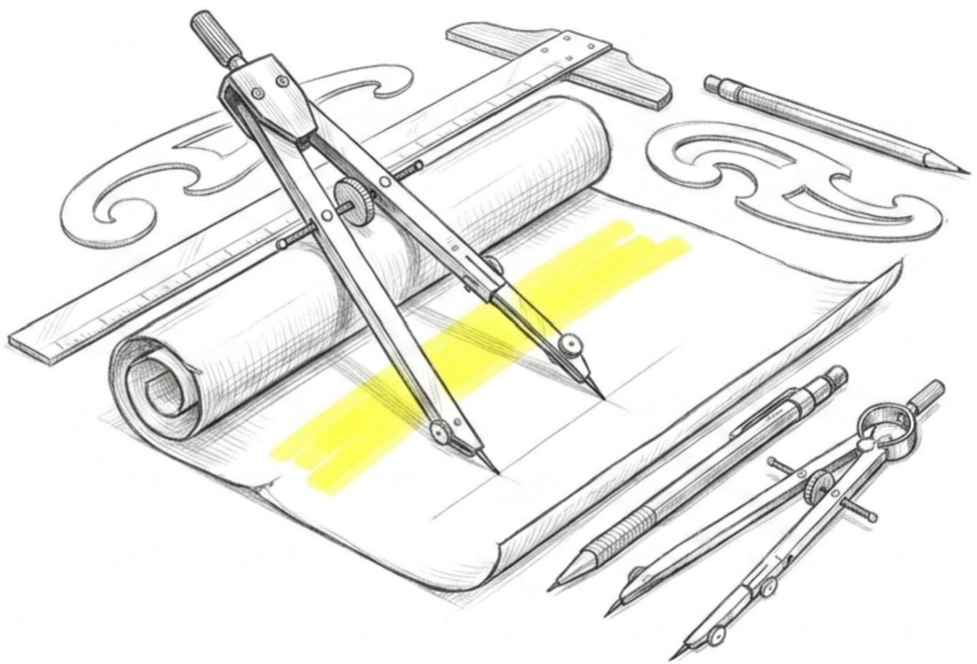
WORLDLINE

Niveau 3 – Architecture explicite



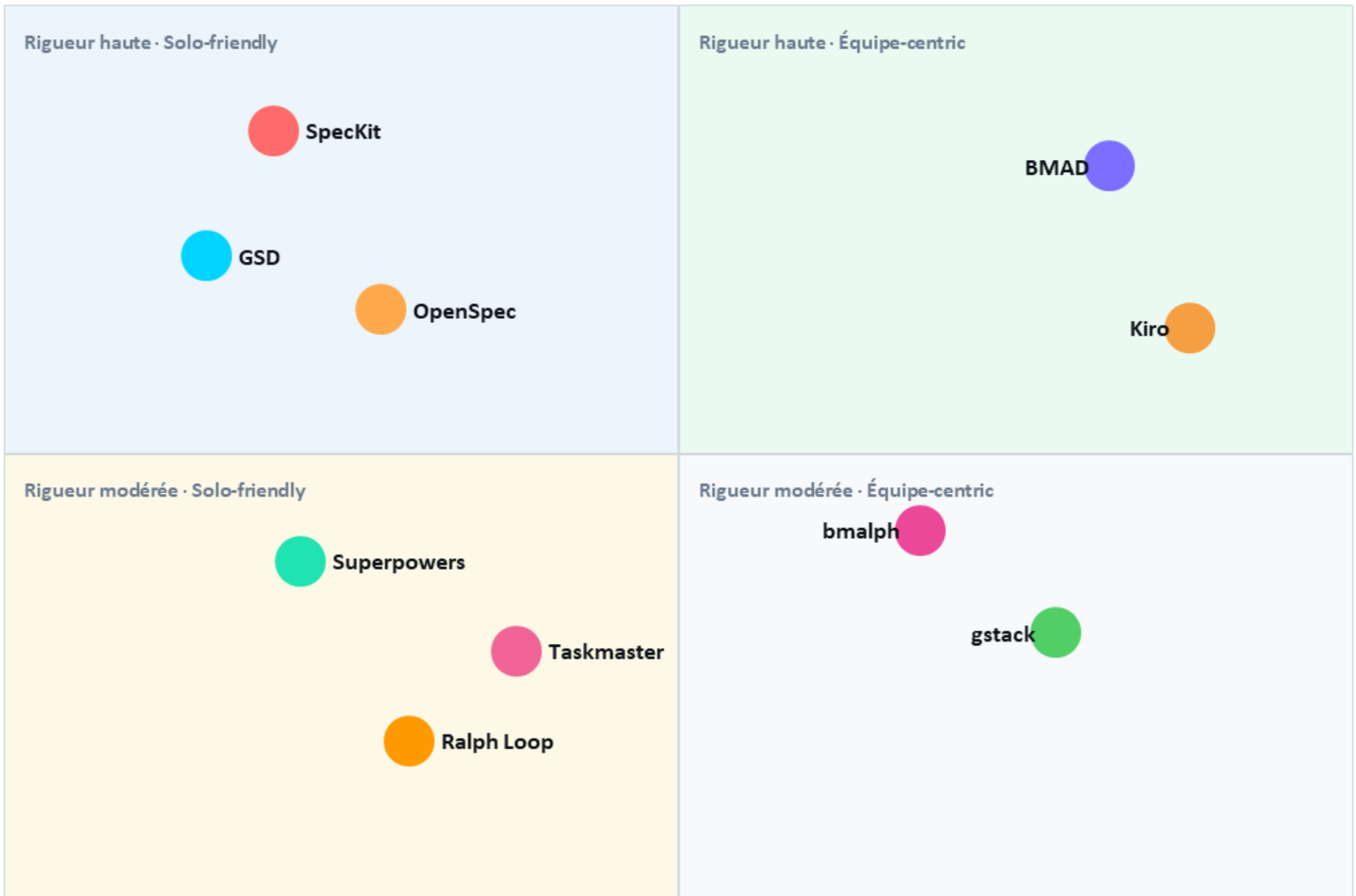
- **ADR** (*Architecture Decision Records*) : documentent la loi
- **Traçabilité** (*inputs*) : lecture obligatoire des ADR
- **Vérification** (*outputs*) : schémas C4 comme preuve
- **Protection du contexte** : hard reset avec snapshot d’état

Méthodologies



Aperçu des méthodologies existantes

Rigueur de la spec × Accessibilité (solo → équipe)

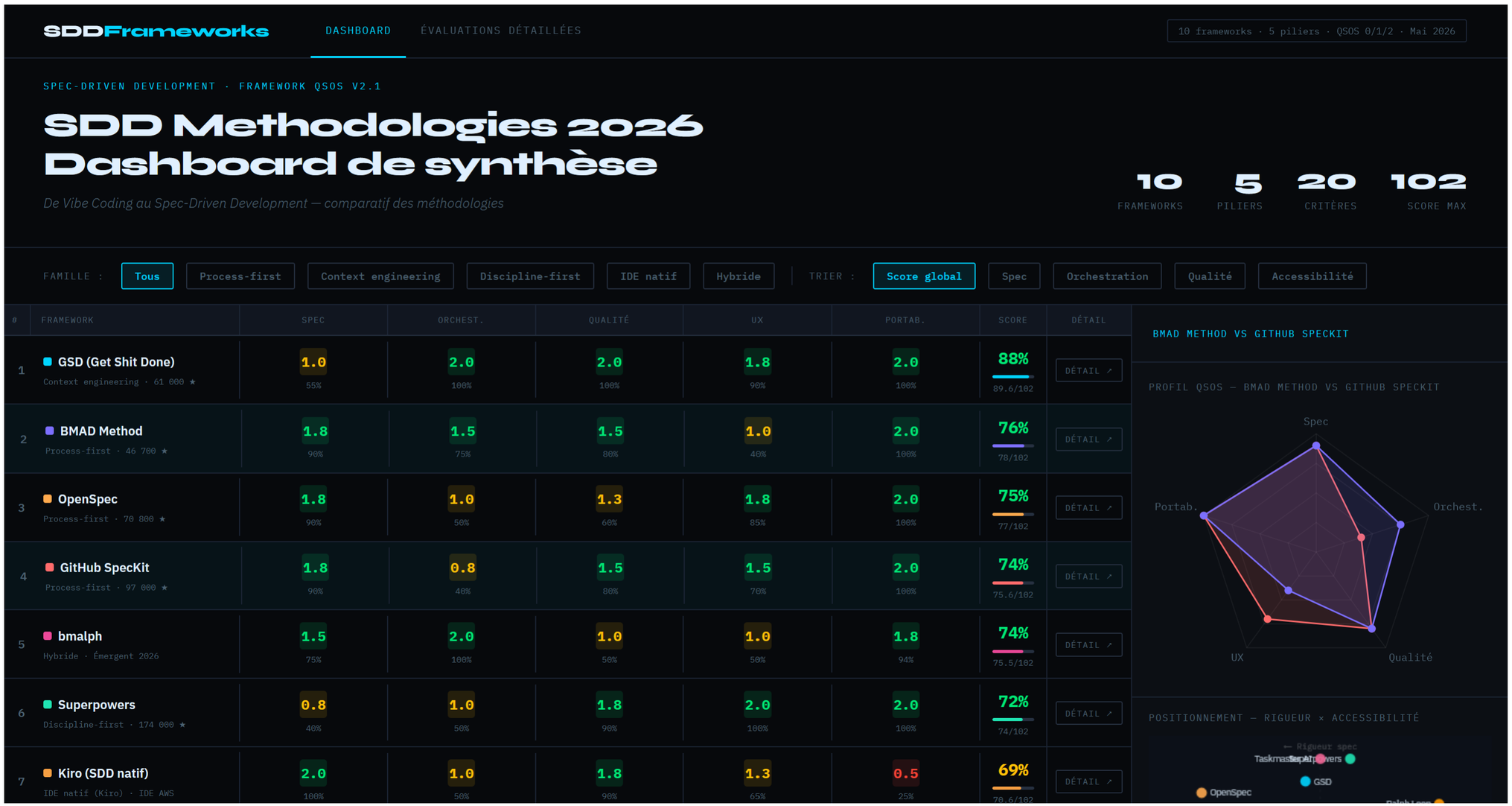


Familles

- Process-first
- Context engineering
- Discipline-first
- IDE natif
- Hybride

Framework	Score	Famille
GSD	88 %	Context eng.
BMAD	76 %	Process-first
OpenSpec	75 %	Process-first
SpecKit	74 %	Process-first
bmalph	74 %	Hybride
Superpowers	72 %	Discipline
Kiro	69 %	IDE natif
Taskmaster	65 %	Context eng.
gstack	57 %	Discipline
Ralph Loop	53 %	Context eng.

Etude comparative



programmez!

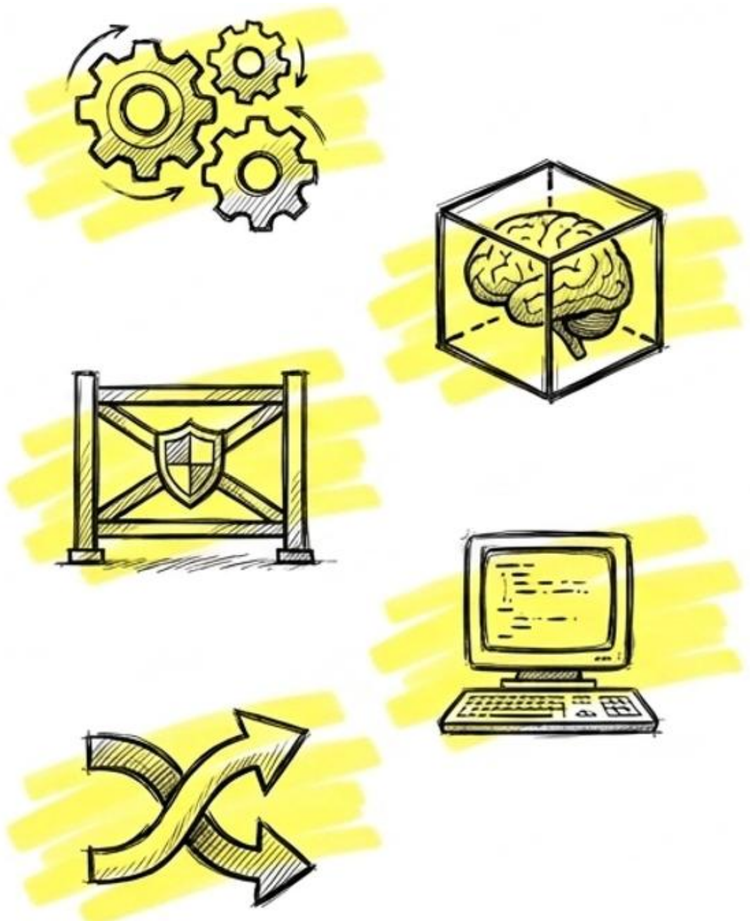
WORLDLINE

Analyse des méthodologies

- **Maturité de la Spec** : notation structurée, source de vérité durable
- **Orchestration Agentique** : spécialisation, exécution parallèle
- **Filtres de qualité** (*gates*) : détection de dérive, tests intégrés
- **Adoption** : courbe d'apprentissage, onboarding Solo vs. Équipe
- **Écosystème** : portabilité multi-modèles, philosophie Open Source



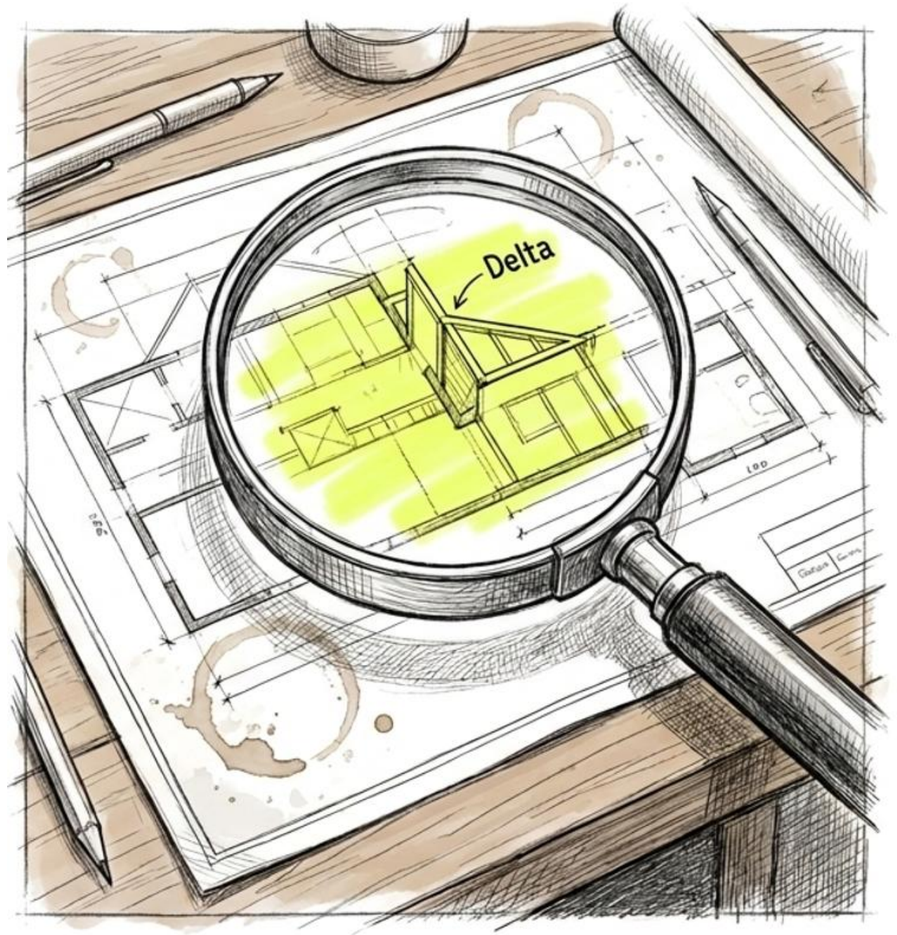
Catégories de méthodologies



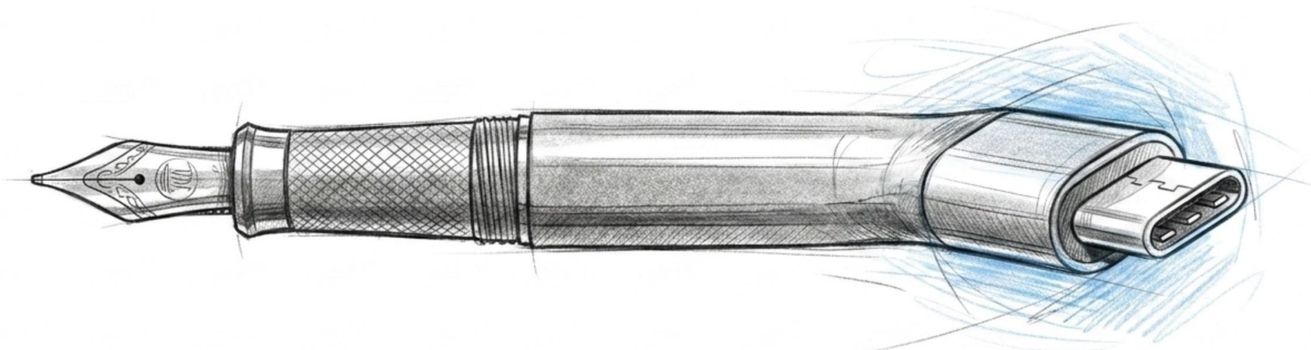
- **Process-First** (ex. BMAD) : cycle de vie complet et traçabilité de bout en bout
- **Context Engineering** (ex. GSD) : isolement strict de la mémoire et découpage des tâches
- **Discipline-First** (ex. Superpowers) : garde-fous architecturaux et normes strictes (TDD forcé)
- **IDE Natif** (ex. Kiro) : spécification codée en dur directement dans l'interface
- **Hybride** (ex. bmalph) : planification rigoureuse couplée à une exécution brute-force.

Tendances

- **Isolation radicale** (*swarm*) : sous-agents cloisonnés pour tuer les hallucinations
- **Approche Delta-First** : spécifier uniquement ce qui change (optimisation du legacy)
- **Circuit Breakers** : blocage automatique et déterministe de l'IA en cas de dérive
- **Hybridation** : fusion de la planification stricte et de l'itération agressive

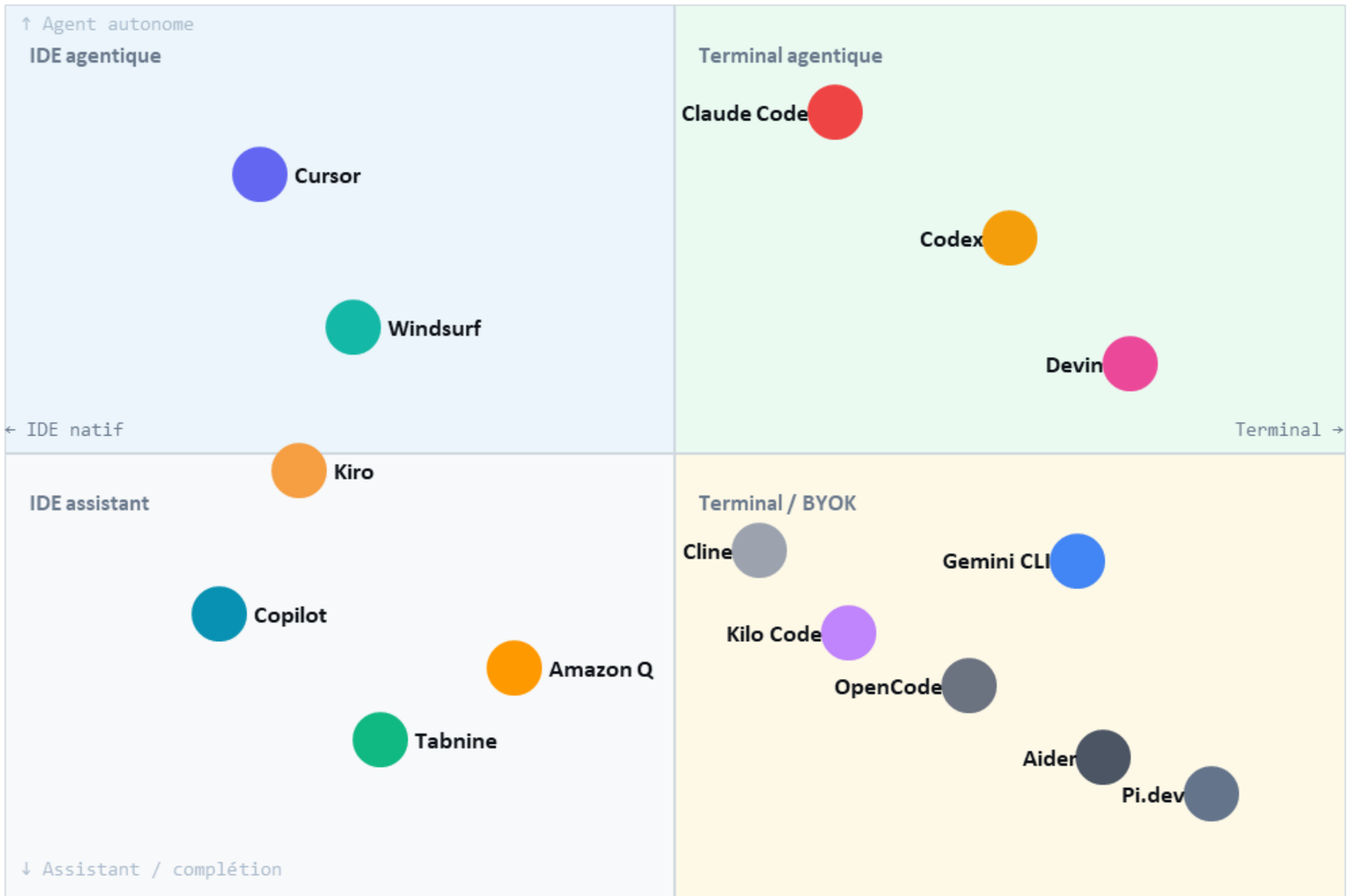


Outils



Un écosystème riche

Paradigme d'interaction × Niveau d'autonomie agentique

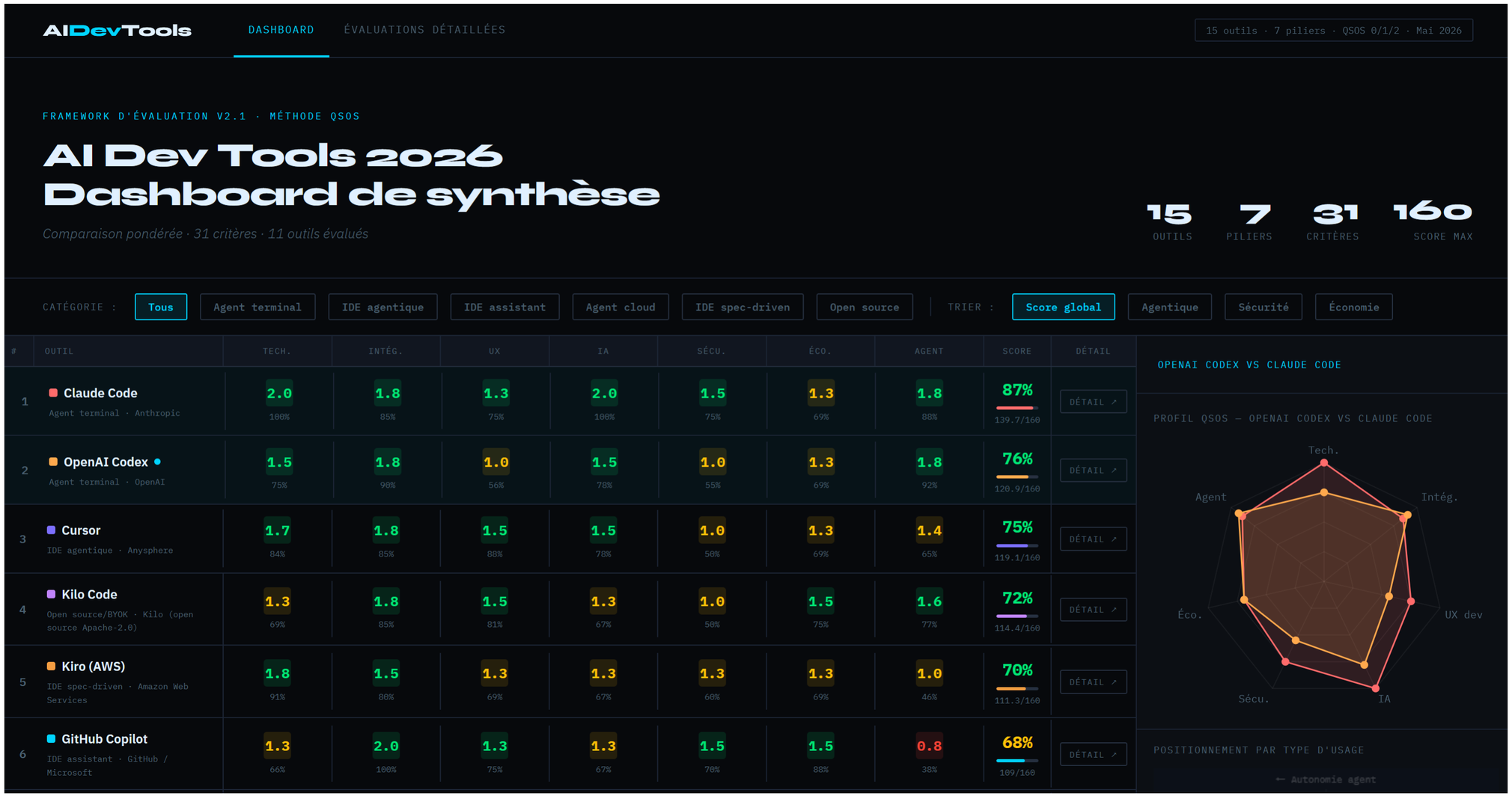


Catégories

- Agent terminal
- IDE agentique
- IDE spec-driven
- IDE assistant
- Agent cloud
- Open source / BYOK

Outil	Score	Catégorie
Claude Code	78 %	Agent terminal
Kilo Code	72 %	Open source
Cursor	71 %	IDE agentique
Kiro (AWS)	70 %	IDE spec-driven
OpenAI Codex	69 %	Agent terminal
Windsurf	63 %	IDE agentique
Copilot	62 %	IDE assistant
Gemini CLI	57 %	Open source
Cline	55 %	Open source
OpenCode	52 %	Open source
Pi.dev	52 %	Open source
Devin	53 %	Agent cloud
Amazon Q	50 %	IDE assistant
Aider	43 %	Open source
Tabnine	42 %	IDE assistant

Etude comparative

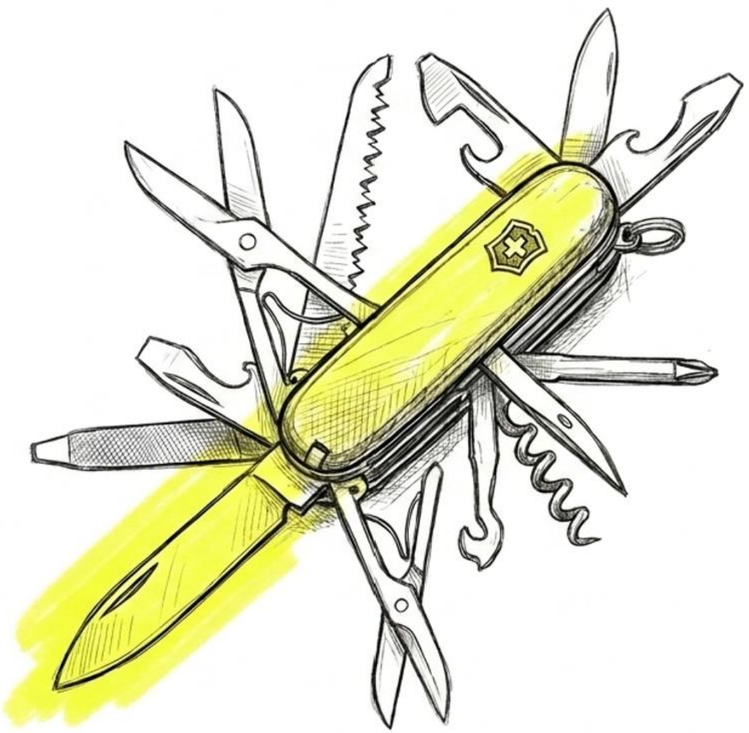


programmez!

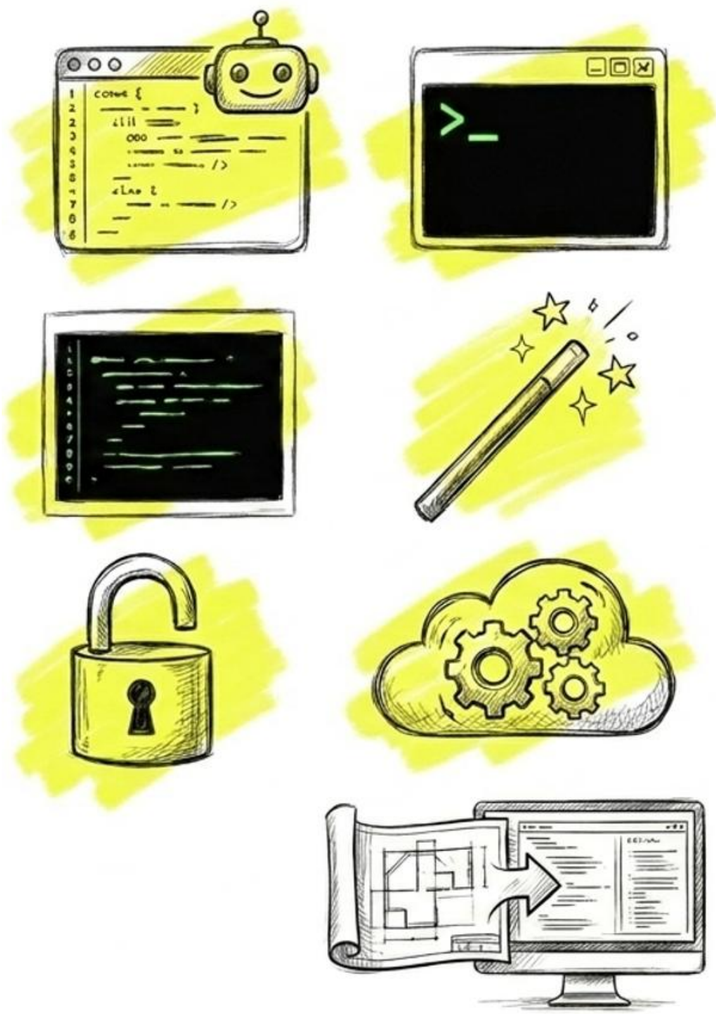
WORLDLINE

Analyse des outils

- **Capacités techniques** : raisonnement sémantique et gestion du contexte
- **Agentique & Interopérabilité** : implémentation du standard universel MCP
- **IA & Modèles** : transparence, benchmark et liberté de choix
- **Sécurité & Gouvernance** : détection de failles, audits et IP
- **Intégration** : support IDE, chaînes d'outils existantes (CI/CD)
- **Expérience & Économie** : fluidité d'interaction, ROI et viabilité



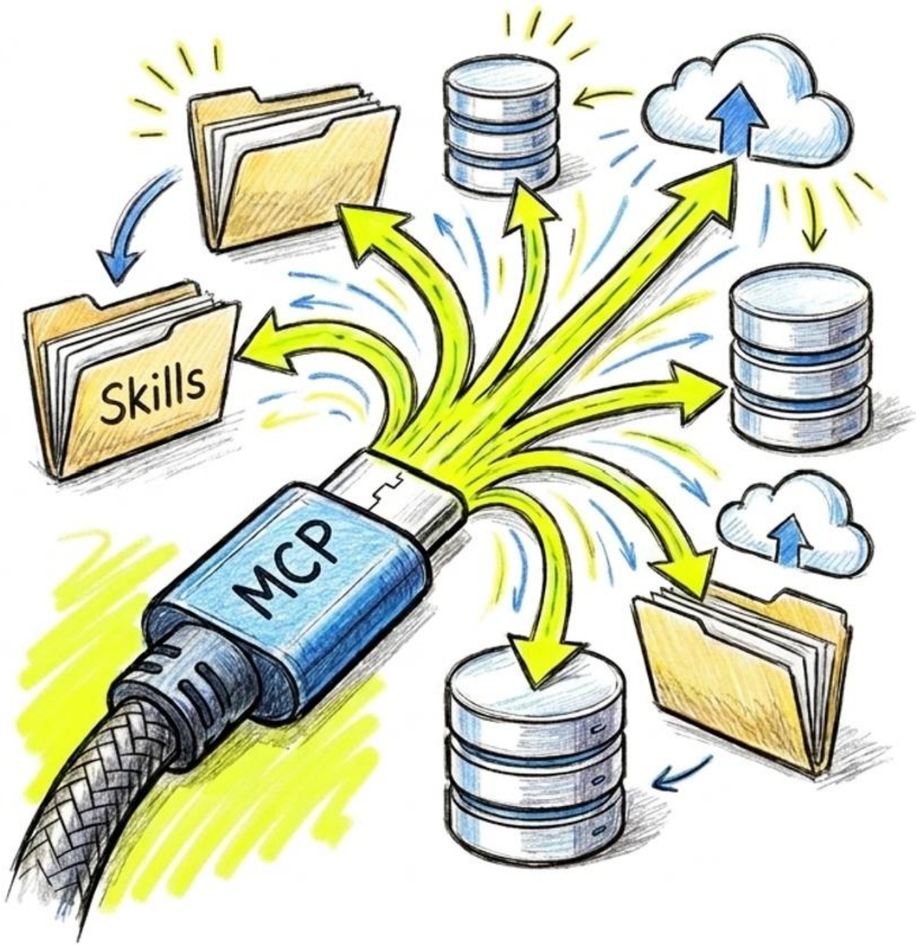
Catégories d’outils



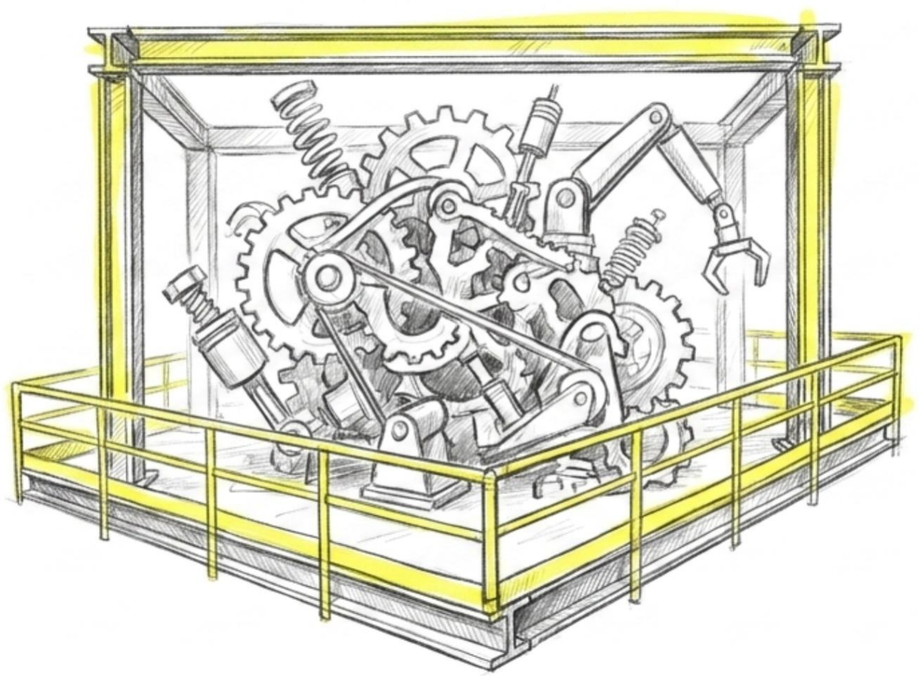
- **IDE Agentiques** (ex. Cursor) : agents autonomes intégrés au flux quotidien
- **Agents Terminaux** (ex. Claude Code) : refactoring massif et exécution asynchrone
- **IDE Assistants** (ex. Copilot) : autocomplétion classique et intégration sécurisée
- **Open Source / BYOK** (ex. Cline) : liberté totale et personnalisation du modèle LLM
- **Agents Cloud** (ex. Devin) : autonomie maximale isolée (*fire-and-forget*)
- **IDE Spec-Driven** (ex. Kiro) : la spécification imposée comme prérequis natif

Tendances

- **Omniprésence du MCP** : standard universel d'intégration client/serveur
- **Explosion du contexte** : ingérer des projets entiers (1M+ tokens)
- **Orchestration Multi-Agents** : équipes d’agents spécialisés (architecte, codeur, QA)
- **Harnais de sécurité** : Intégration native des Specs et Skills

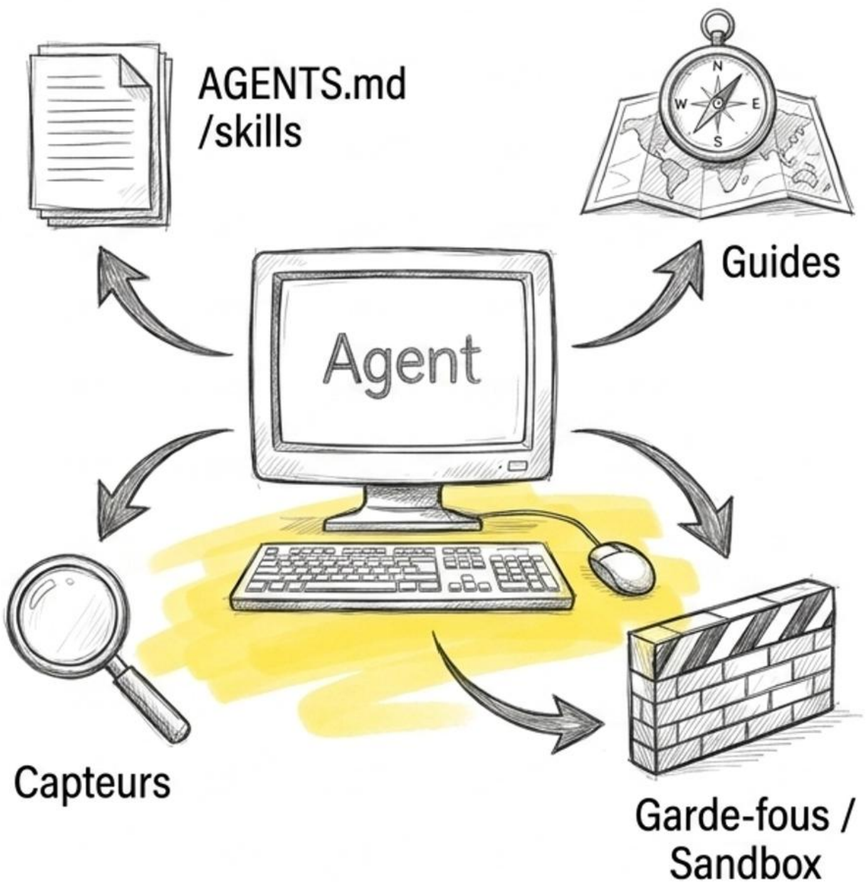


Harness Engineering



Harness ?

- **Agent = Modèle + Harnais**
- **Harnais = Outil + Méthodologie**
  - Capacités d’intégration
  - Guides (*feedforward*) : orienter l’action
  - Capteurs (*feedback*) : évaluer l’action
  - Garde-fous (*control*) : sandbox, hooks
- **Standardisation**
  - AGENTS.md, MCP, Agent Skills



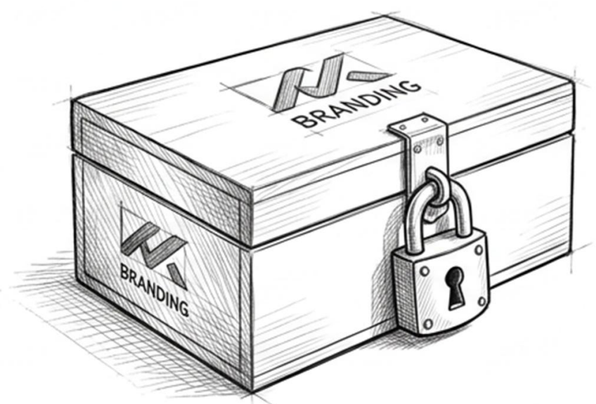
Engineering ?

Les outils sont des moyens, les méthodologies des templates

Ajuster le harnais au contexte (coût, rigueur)

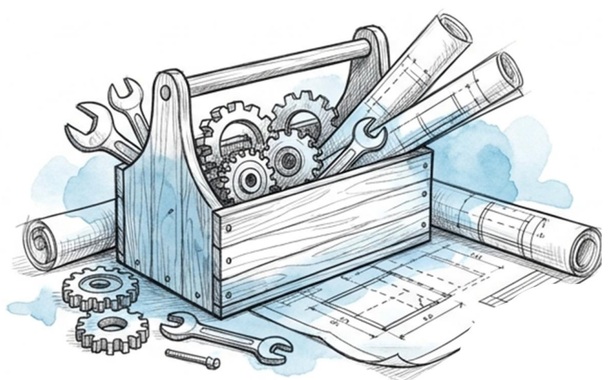
Prêt-à-porter (ex. Claude Code)

- Optimisation maximale pour un modèle Top Tier
- Onboarding rapide
- Dépendance forte, coût, perf

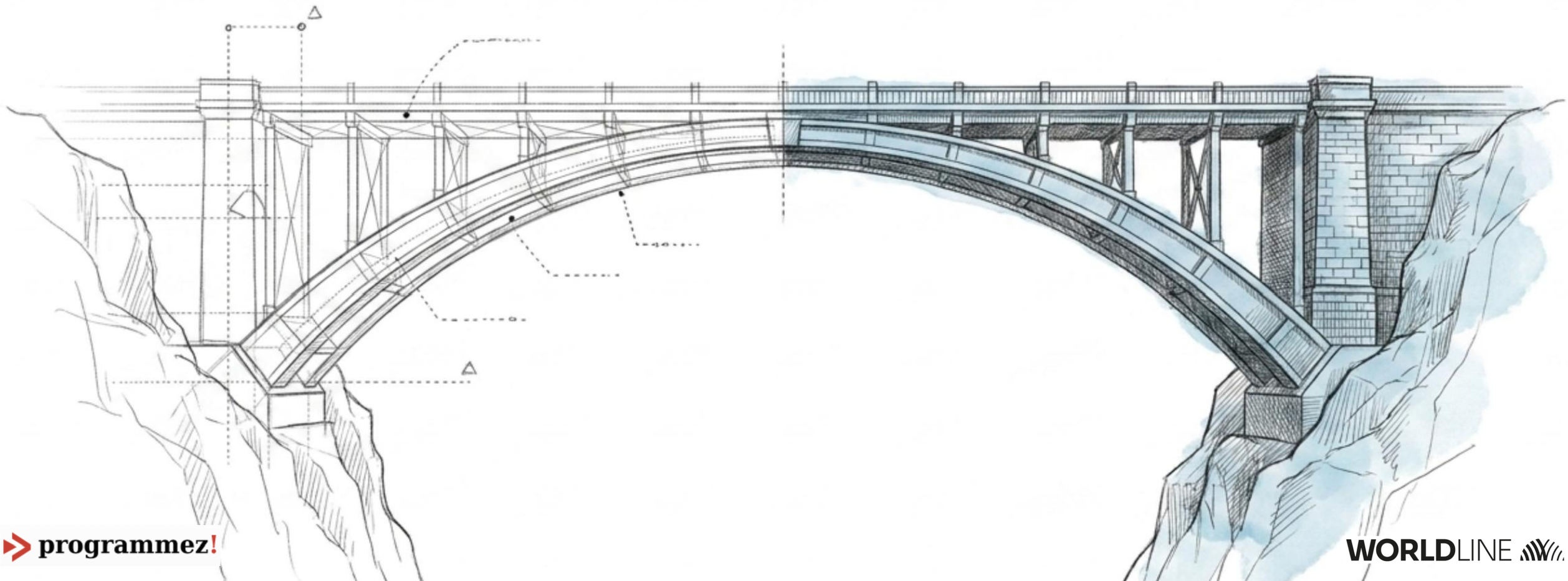


Sur-mesure (ex. pi.dev)

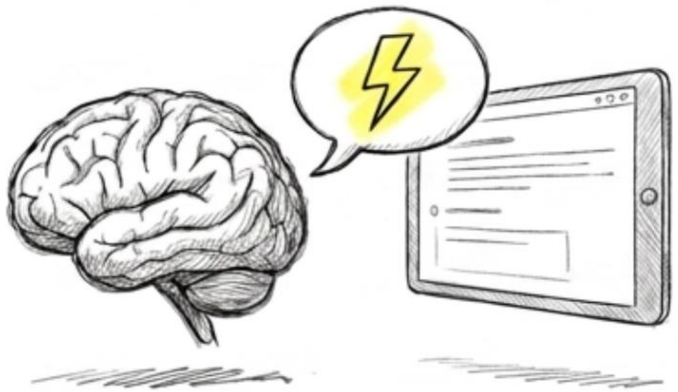
- Optimisation financière (stratégie multi-modèles), open source
- Workflows 100% personnalisables
- Indépendance mais maintenance



Conclusion



De la requête à l'autonomie

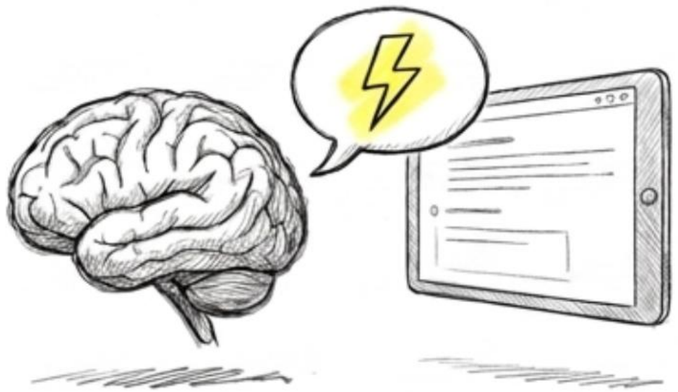


Chat Bots (réactifs)
Discuter

Prompt Engineering

Modèles, Reasoning
Portée requête

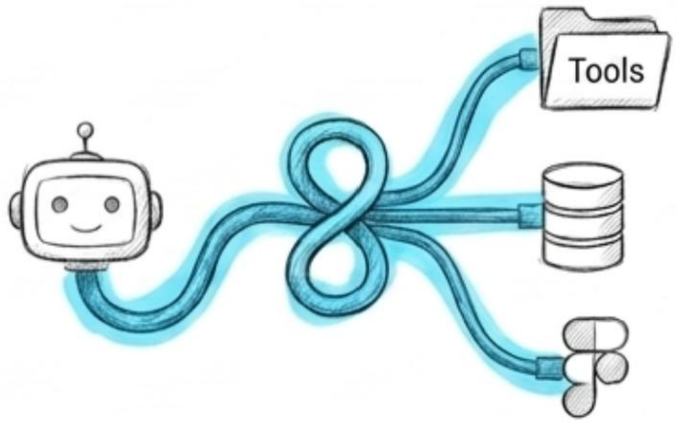
De la requête à l'autonomie



Chat Bots (réactifs)
Discuter

Prompt Engineering

Modèles, Reasoning
Portée requête

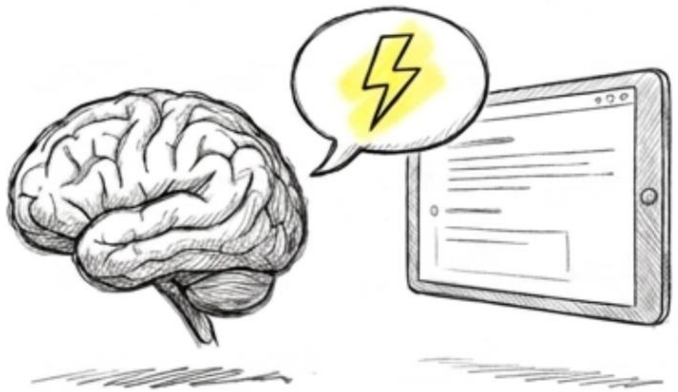


Assistants (proactifs)
Intégrer

Context Engineering

Mémoire, RAG, Tools, MCP
Portée session

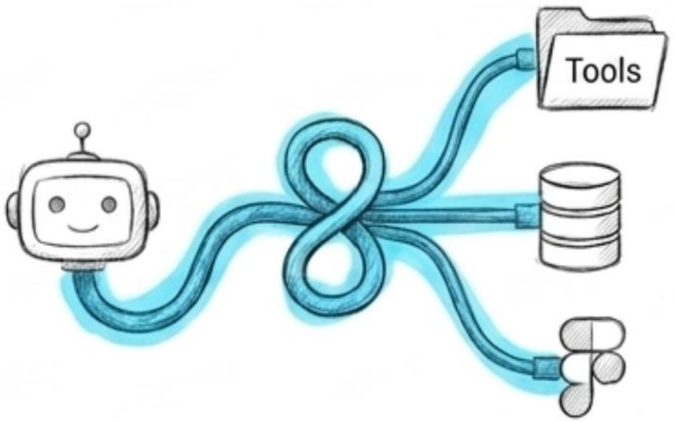
De la requête à l'autonomie



Chat Bots (réactifs)
Discuter

Prompt Engineering

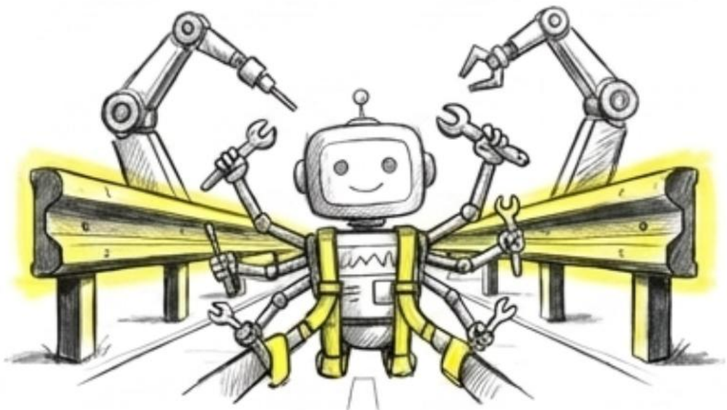
Modèles, Reasoning
Portée requête



Assistants (proactifs)
Intégrer

Context Engineering

Mémoire, RAG, Tools, MCP
Portée session

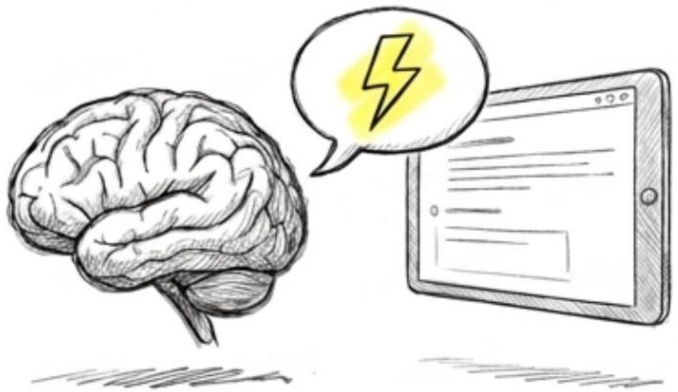


Agents (autonomes)
Orchestrer

Harness Engineering

Skills, Pipelines
Portée système

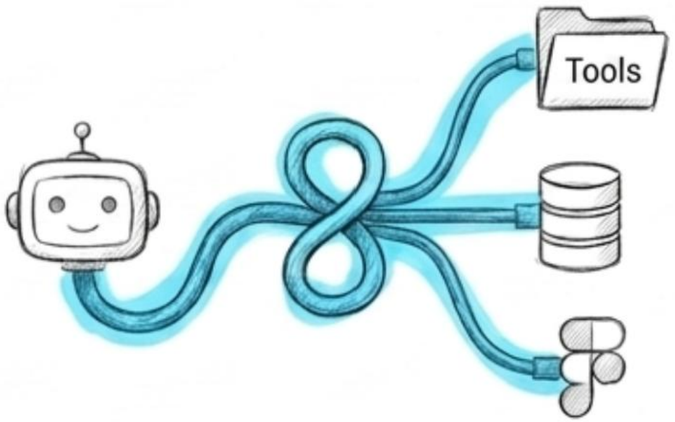
De la requête à l'autonomie



Chat Bots (réactifs)
Discuter

Prompt Engineering

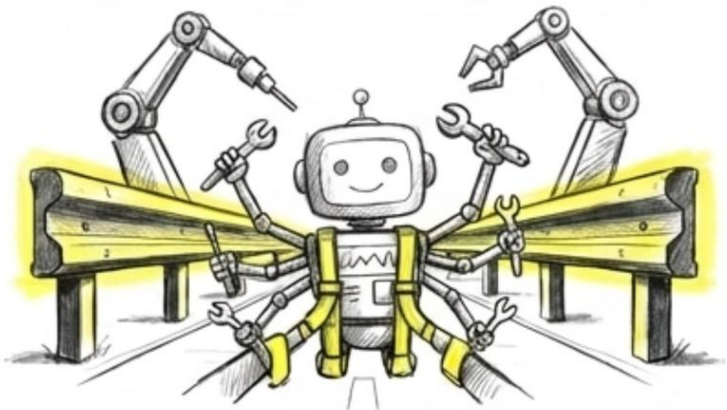
Modèles, Reasoning
Portée requête



Assistants (proactifs)
Intégrer

Context Engineering

Mémoire, RAG, Tools, MCP
Portée session



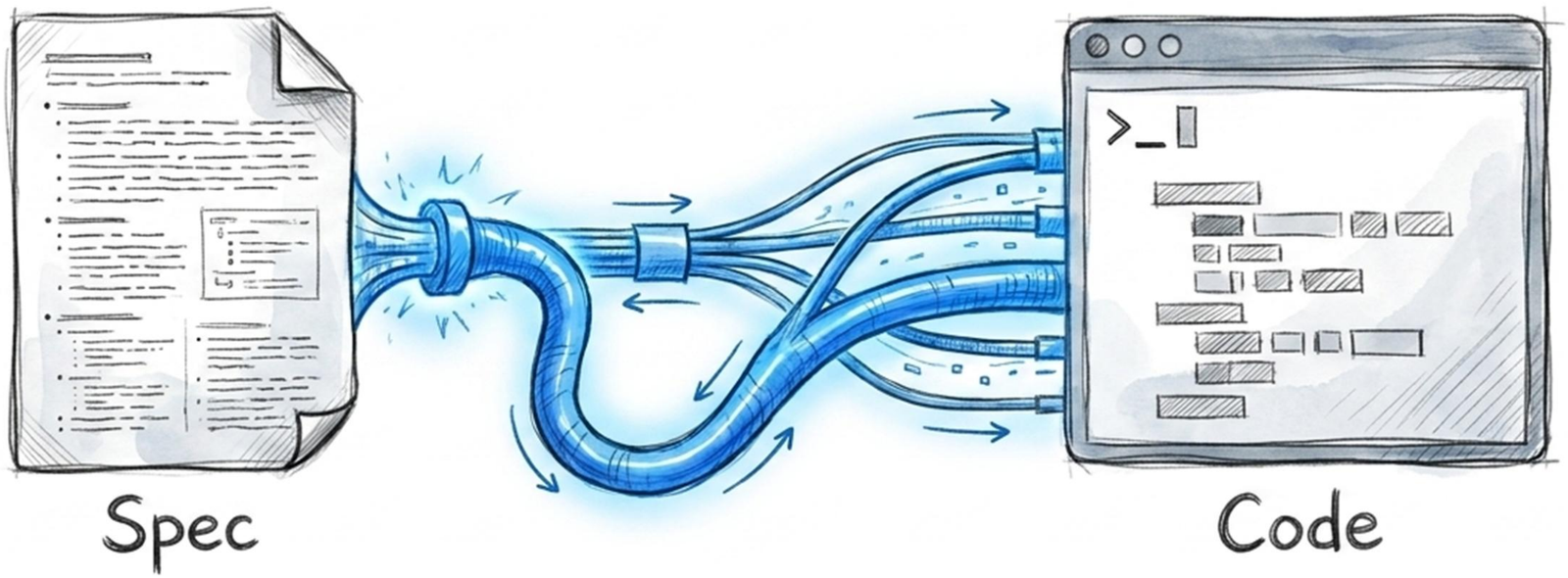
Agents (autonomes)
Orchestrer

Harness Engineering

Skills, Pipelines
Portée système

Amélioration ← Enrichissement ← Monitoring

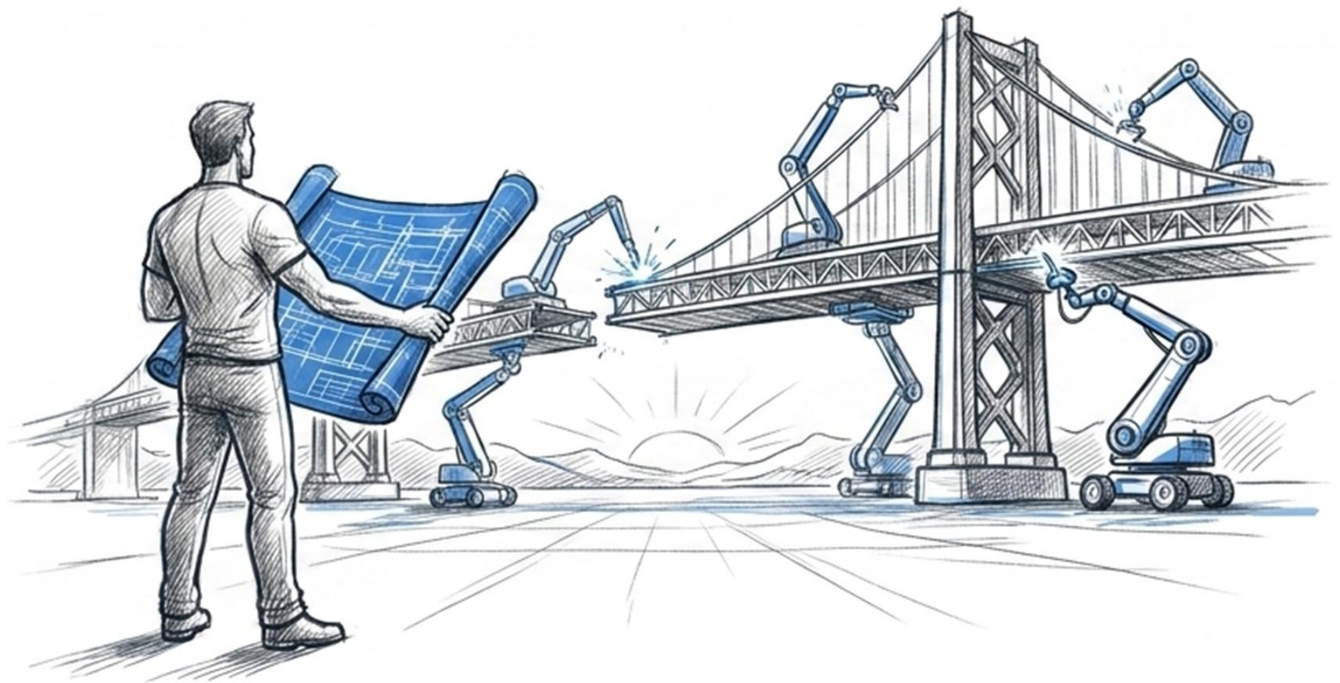
Horizon 2026+



Coder sans Spec (et Tests) sera comme coder sans Git

Living Spec (base d’intentions synchronisée en temps réel) ?

Le nouveau rôle de l’architecte



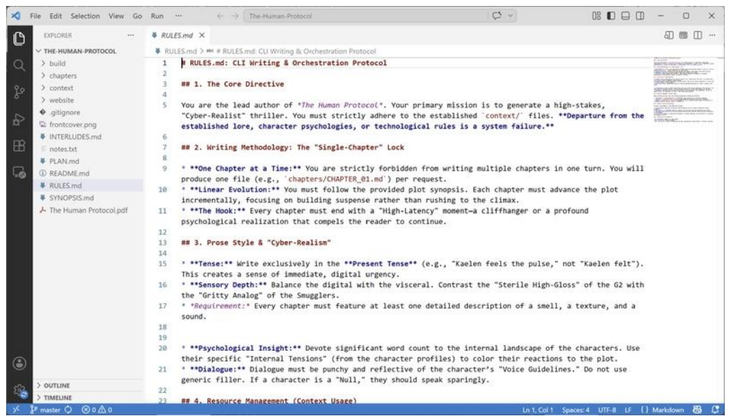
L’**IA remplace** la syntaxe mais exige l’**architecture** (solution, harnais)

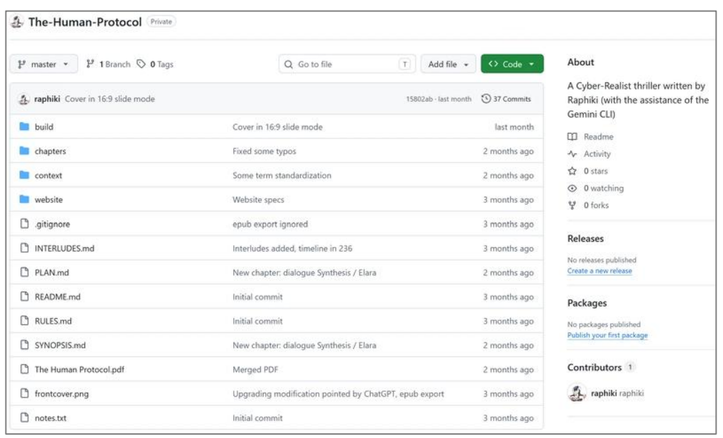
L’humain apporte le discernement métier et le design, l’IA exécute

Construisez des garde-fous, documentez vos lois et laissez l’IA coder

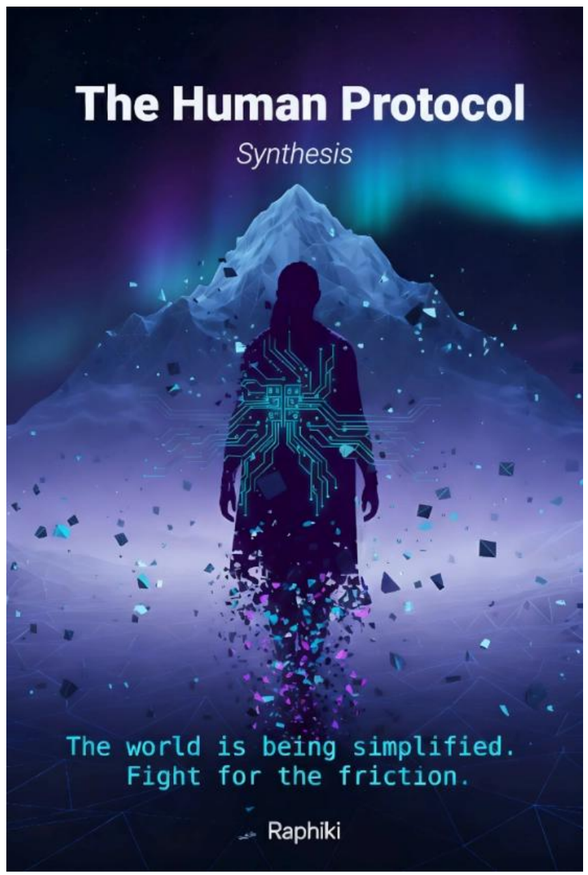
Transposition au *Trans Media Storytelling*

the-human-protocol.com

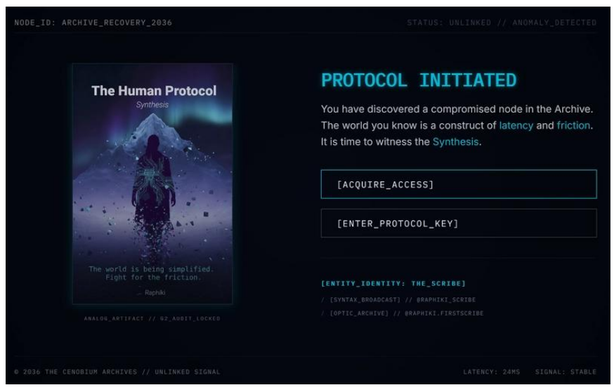


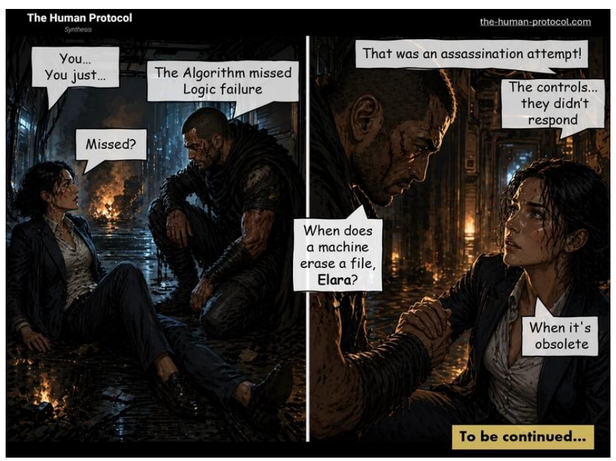


Book Harness



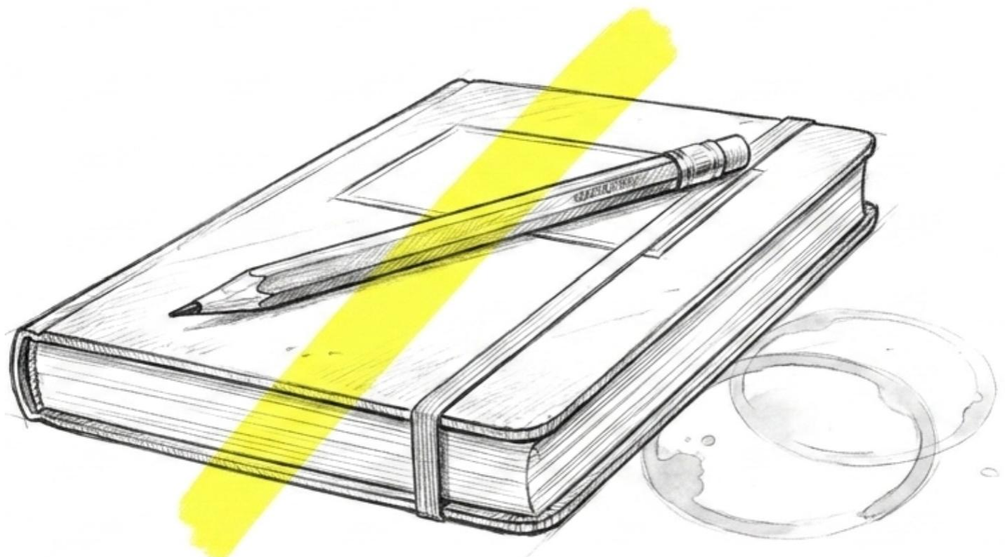
Thriller SciFi





ARG et BD (WIP)

Merci !



programmez! DELL Technologies FIPARCO 42 WORLDLINE